In [1]:
```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

In [2]:
```python
data = pd.read_excel('Labmentix_first_raw.xlsx')
```

In [3]:
```python
data
```

Out[3]:

	ID	Marital Status	Gender	Income	Children	Education	Occupation	Home Owner	Cars	Commute Distance	Region	Age	Purchased Bike
0	12496	M	F	40000	1	Bachelors	Skilled Manual	Yes	0	0-1 Miles	Europe	42	No
1	24107	M	M	30000	3	Partial College	Clerical	Yes	1	0-1 Miles	Europe	43	No
2	14177	M	M	80000	5	Partial College	Professional	No	2	2-5 Miles	Europe	60	No
3	24381	S	M	70000	0	Bachelors	Professional	Yes	1	5-10 Miles	Pacific	41	Yes
4	25597	S	M	30000	0	Bachelors	Clerical	No	0	0-1 Miles	Europe	36	Yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...
1021	16466	S	F	20000	0	Partial High School	Manual	No	2	0-1 Miles	Europe	32	Yes
1022	19273	M	F	20000	2	Partial College	Manual	Yes	0	0-1 Miles	Europe	63	No
1023	22400	M	M	10000	0	Partial College	Manual	No	1	0-1 Miles	Pacific	26	Yes
1024	20942	S	F	20000	0	High School	Manual	No	1	5-10 Miles	Europe	31	No
1025	18484	S	M	80000	2	High School	Skilled Manual	No	2	1-2 Miles	Pacific	50	Yes

1026 rows × 13 columns

In [4]:
```python
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1026 entries, 0 to 1025
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype 
---  ------            --------------  ----- 
 0   ID                1026 non-null   int64 
 1   Marital Status    1026 non-null   object
 2   Gender            1026 non-null   object
 3   Income            1026 non-null   int64 
 4   Children          1026 non-null   int64 
 5   Education         1026 non-null   object
 6   Occupation        1026 non-null   object
 7   Home Owner        1026 non-null   object
 8   Cars              1026 non-null   int64 
 9   Commute Distance  1026 non-null   object
 10  Region            1026 non-null   object
 11  Age               1026 non-null   int64 
 12  Purchased Bike    1026 non-null   object
dtypes: int64(5), object(8)
memory usage: 104.3+ KB
```

In [5]:
```python
dupli_exi = data.duplicated().any()
print(f"Duplicate exist in data: {dupli_exi}")
dupli_count = data.duplicated().sum()
print(f"Duplicate count: {dupli_count}")
```

```
Duplicate exist in data: True
Duplicate count: 26
```

In [6]:
```python
all_dupli = data[data.duplicated(keep=False)]
all_dupli_sort = all_dupli.sort_values(by=data.columns.tolist())
print(f"Total duplicate rows: {len(all_dupli)}")
print(f"Number of duplicated rows:{len(all_dupli)-data.duplicated().sum()}")
```

```
Total duplicate rows: 52
Number of duplicated rows:26
```

In [7]:
```python
all_dupli_sort.head(20)
```

Out[7]:

	ID	Marital Status	Gender	Income	Children	Education	Occupation	Home Owner	Cars	Commute Distance	Region	Age	Purchased Bike
12	11434	M	M	170000	5	Partial College	Professional	Yes	0	0-1 Miles	Europe	55	No
1004	11434	M	M	170000	5	Partial College	Professional	Yes	0	0-1 Miles	Europe	55	No
25	12590	S	M	30000	1	Bachelors	Clerical	Yes	0	0-1 Miles	Europe	63	No
1017	12590	S	M	30000	1	Bachelors	Clerical	Yes	0	0-1 Miles	Europe	63	No
17	12610	M	F	30000	1	Bachelors	Clerical	Yes	0	0-1 Miles	Europe	47	No
1009	12610	M	F	30000	1	Bachelors	Clerical	Yes	0	0-1 Miles	Europe	47	No
11	12697	S	F	90000	0	Bachelors	Professional	No	4	10+ Miles	Pacific	36	No
1003	12697	S	F	90000	0	Bachelors	Professional	No	4	10+ Miles	Pacific	36	No
5	13507	M	F	10000	2	Partial College	Manual	Yes	0	1-2 Miles	Europe	50	No
1000	13507	M	F	10000	2	Partial College	Manual	Yes	0	1-2 Miles	Europe	50	No
29	16466	S	F	20000	0	Partial High School	Manual	No	2	0-1 Miles	Europe	32	Yes
1021	16466	S	F	20000	0	Partial High School	Manual	No	2	0-1 Miles	Europe	32	Yes
26	17841	S	M	30000	0	Partial College	Clerical	No	1	0-1 Miles	Europe	29	Yes
1018	17841	S	M	30000	0	Partial College	Clerical	No	1	0-1 Miles	Europe	29	Yes
27	18283	S	F	100000	0	Bachelors	Professional	No	1	5-10 Miles	Pacific	40	No
1019	18283	S	F	100000	0	Bachelors	Professional	No	1	5-10 Miles	Pacific	40	No
28	18299	M	M	70000	5	Partial College	Skilled Manual	Yes	2	5-10 Miles	Pacific	44	No
1020	18299	M	M	70000	5	Partial College	Skilled Manual	Yes	2	5-10 Miles	Pacific	44	No
33	18484	S	M	80000	2	High School	Skilled Manual	No	2	1-2 Miles	Pacific	50	Yes
1025	18484	S	M	80000	2	High School	Skilled Manual	No	2	1-2 Miles	Pacific	50	Yes

In [8]:
```python
data_cleaned = data.drop_duplicates(keep='first')
```

In [9]:
```python
data.shape
```

Out[9]:
```
(1026, 13)
```

In [10]:
```python
data_cleaned.shape   #Found 26 duplicated rows, now they will be dropped
```

Out[10]:
```
(1000, 13)
```

In [11]:
```python
data_cleaned
```

Out[11]:

	ID	Marital Status	Gender	Income	Children	Education	Occupation	Home Owner	Cars	Commute Distance	Region	Age	Purchased Bike
0	12496	M	F	40000	1	Bachelors	Skilled Manual	Yes	0	0-1 Miles	Europe	42	No
1	24107	M	M	30000	3	Partial College	Clerical	Yes	1	0-1 Miles	Europe	43	No
2	14177	M	M	80000	5	Partial College	Professional	No	2	2-5 Miles	Europe	60	No
3	24381	S	M	70000	0	Bachelors	Professional	Yes	1	5-10 Miles	Pacific	41	Yes
4	25597	S	M	30000	0	Bachelors	Clerical	No	0	0-1 Miles	Europe	36	Yes
...	...	...	...	...	...	...	...	...	...	...	...	...	...
995	23731	M	M	60000	2	High School	Professional	Yes	2	2-5 Miles	North America	54	Yes
996	28672	S	M	70000	4	Graduate Degree	Professional	Yes	0	2-5 Miles	North America	35	Yes
997	11809	M	M	60000	2	Bachelors	Skilled Manual	Yes	0	0-1 Miles	North America	38	Yes
998	19664	S	M	100000	3	Bachelors	Management	No	3	1-2 Miles	North America	38	No
999	12121	S	M	60000	3	High School	Professional	Yes	2	10+ Miles	North America	53	Yes

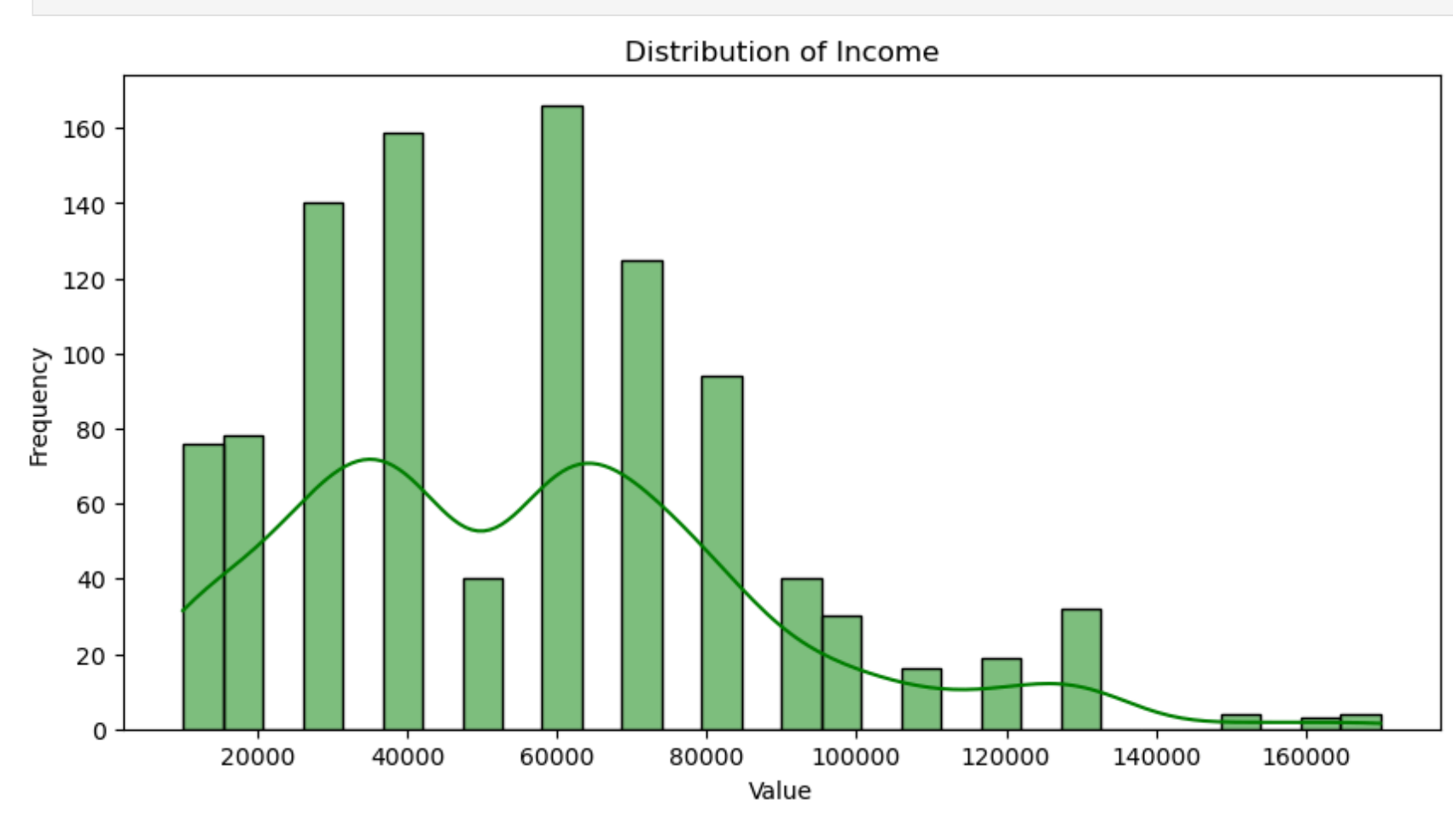
1000 rows × 13 columns

In [12]:
```python
data_cleaned.describe()
```

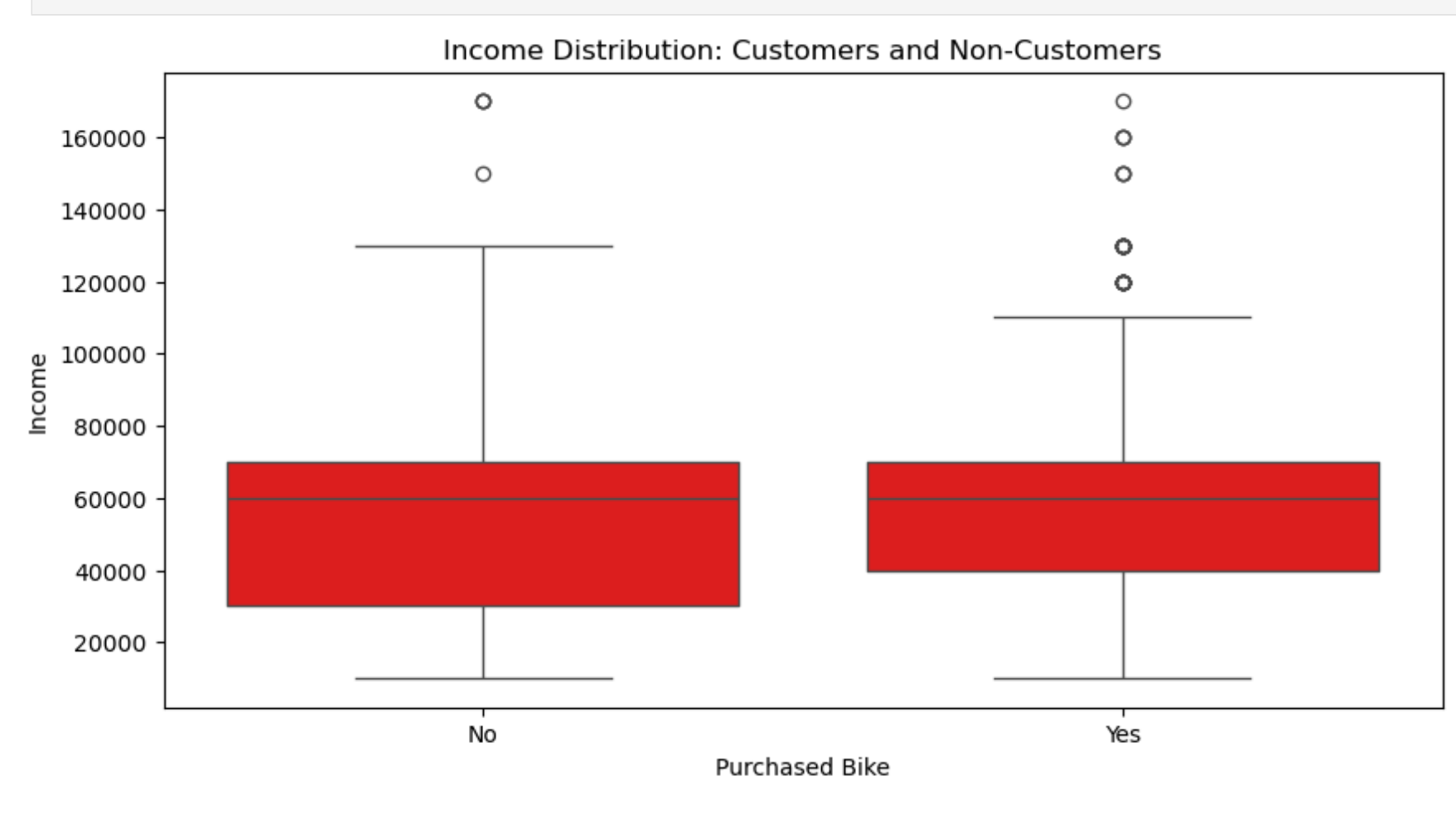
Out[12]:

	ID	Income	Children	Cars	Age
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	19965.992000	56360.000000	1.898000	1.442000	44.163000
std	5347.333948	31085.635215	1.628572	1.125123	11.364488
min	11000.000000	10000.000000	0.000000	0.000000	25.000000
25%	15290.750000	30000.000000	0.000000	1.000000	35.000000
50%	19744.000000	60000.000000	2.000000	1.000000	43.000000
75%	24470.750000	70000.000000	3.000000	2.000000	52.000000
max	29447.000000	170000.000000	5.000000	4.000000	89.000000

In [13]:
```python
plt.figure(figsize=(10,5))
sns.histplot(data,x='Income',kde=True,bins=30,color="green")
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.title(f"Distribution of Income")
plt.show()
```



In [14]:
```python
plt.figure(figsize=(10,5))
sns.boxplot(data,x='Purchased Bike',y='Income',color='red')
plt.title('Income Distribution: Customers and Non-Customers')
plt.xlabel('Purchased Bike')
plt.ylabel('Income')
plt.show()
```



In [15]:
```python
dupli_exi = data.duplicated().any()
print(f"Duplicate exist in data: {dupli_exi}")
dupli_count = data.duplicated().sum()
print(f"Duplicate count: {dupli_count}")
```

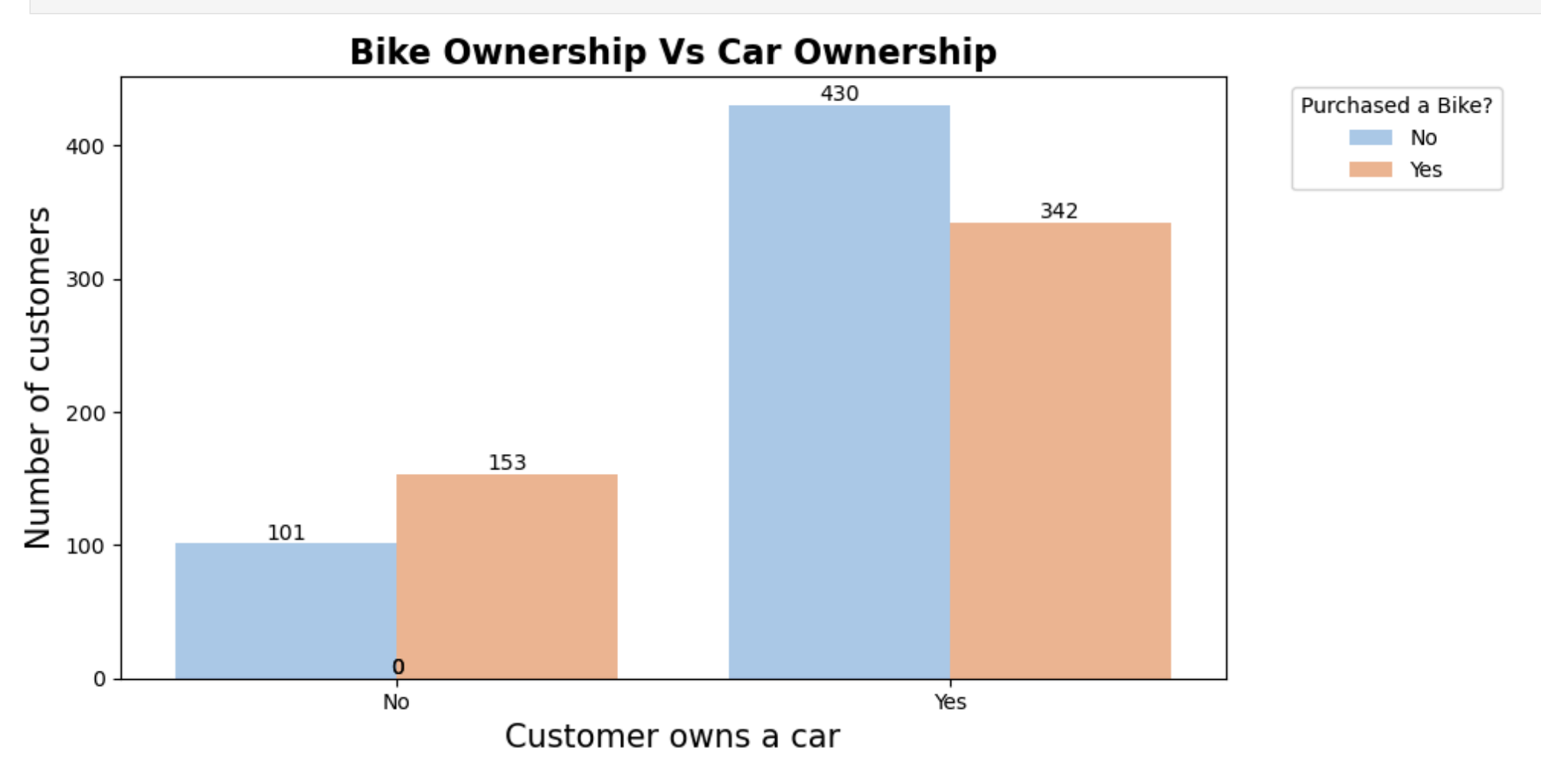
```
Duplicate exist in data: True
Duplicate count: 26
```

In [16]:
```python
data['Has_Car']=data['Cars'].apply(lambda x:'Yes' if x>=1 else 'No')
```

In [17]:
```python
contingency_table=pd.crosstab(data['Has_Car'],data['Purchased Bike'])
print("Comparision of customer groups:")
print(contingency_table)
print("\n")
```

```
Comparision of customer groups:
Purchased Bike   No  Yes
Has_Car                 
No              101  153
Yes             430  342


```

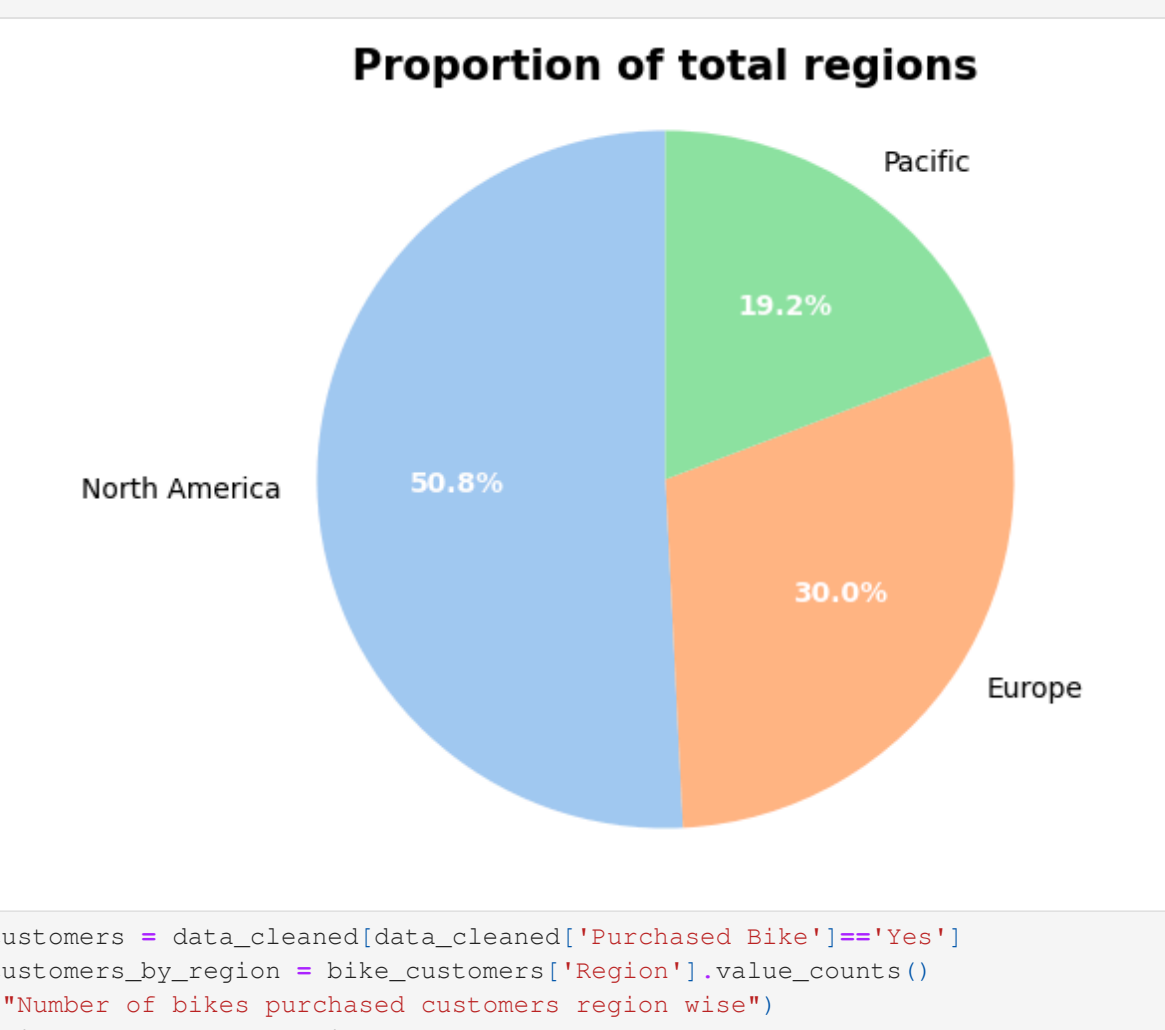
In [18]:
```python
plt.figure(figsize=(10,5))
cp=sns.countplot(data,x='Has_Car',hue='Purchased Bike',palette='pastel')
plt.title("Bike Ownership Vs Car Ownership",fontsize=16,fontweight='bold')
plt.xlabel("Customer owns a car",fontsize=15)
plt.ylabel("Number of customers",fontsize=15)
for x in cp.patches:
    cp.annotate(f'{x.get_height():.0f}', (x.get_x() + x.get_width() / 2., x.get_height()),
                ha='center', va='center', xytext=(0, 5), textcoords='offset points')

plt.legend(title='Purchased a Bike?', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



In [19]:
```python
sales_by_region = data_cleaned['Region'].value_counts()
print(sales_by_region)
```

```
Region
North America    508
Europe           300
Pacific          192
Name: count, dtype: int64
```

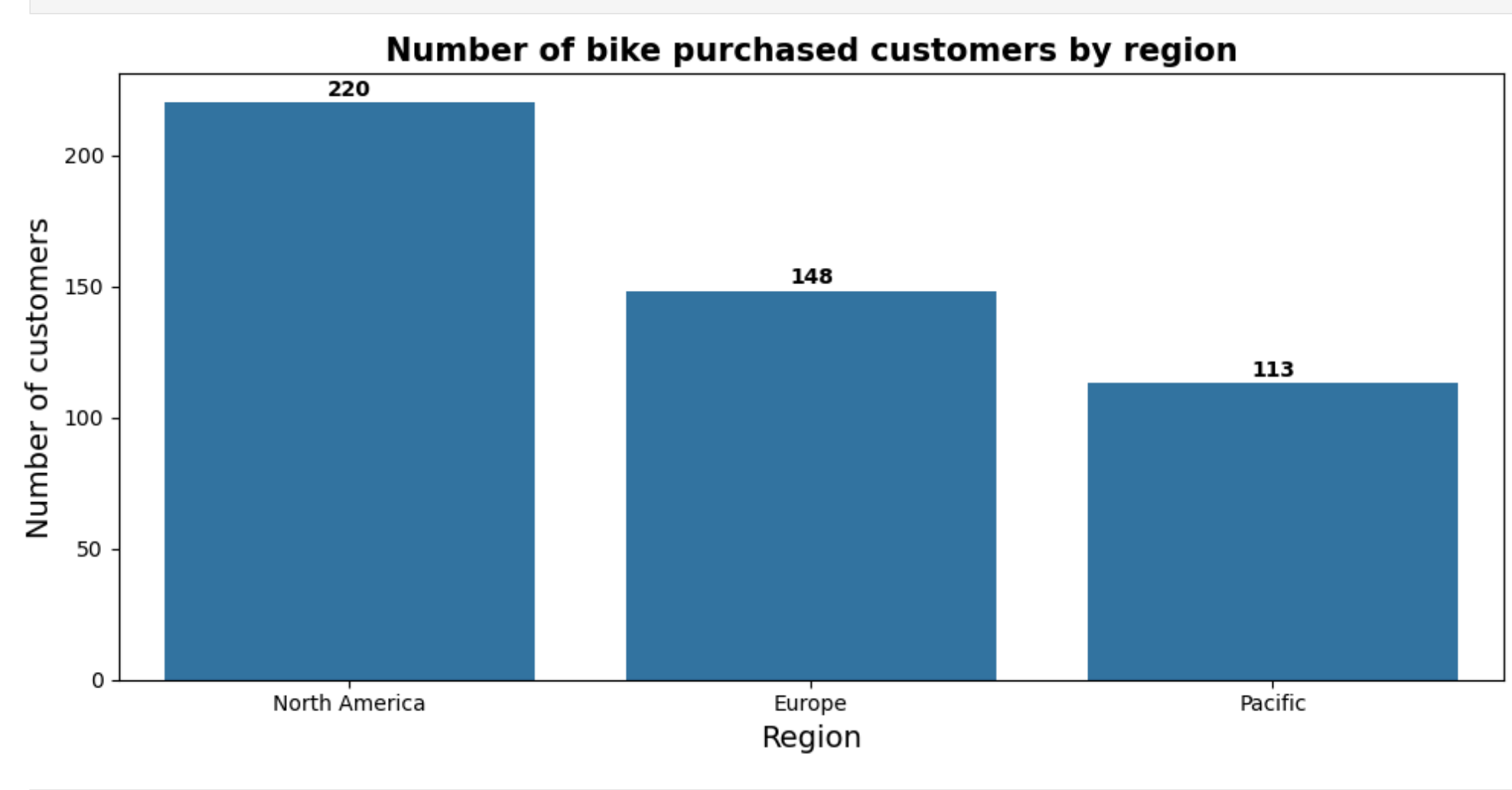
In [20]:
```python
colors = sns.color_palette('pastel')[0:len(sales_by_region)]
plt.figure(figsize=(10,5))
wedges,texts,autotexts = plt.pie(sales_by_region.values,labels=sales_by_region.index,colors=colors,autopct="%1.1f%%",startangle=90)
for autotext in autotexts:
    autotext.set_color('white')
    autotext.set_fontweight('bold')
plt.title("Proportion of total regions",fontsize=15,fontweight='bold')
plt.axis('equal')
plt.show()
```



In [21]:
```python
bike_customers = data_cleaned[data_cleaned['Purchased Bike']=='Yes']
bike_customers_by_region = bike_customers['Region'].value_counts()
print("Number of bikes purchased customers region wise")
print(bike_customers_by_region)
```

```
Number of bikes purchased customers region wise
Region
North America    220
Europe           148
Pacific          113
Name: count, dtype: int64
```

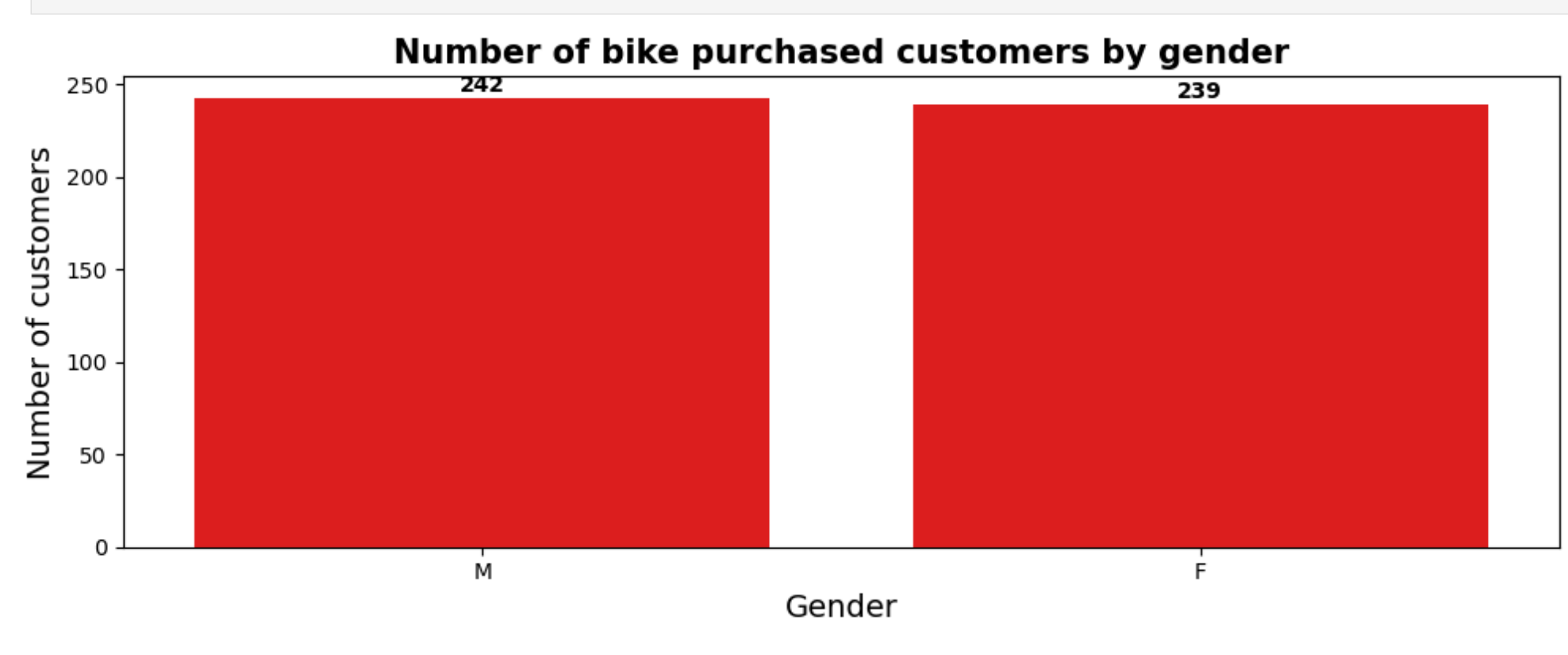
In [22]:
```python
plt.figure(figsize=(10,5))
bg = sns.barplot(x=bike_customers_by_region.index,y=bike_customers_by_region.values)
plt.title("Number of bike purchased customers by region",fontsize=15,fontweight="bold")
plt.xlabel('Region',fontsize=14)
plt.ylabel('Number of customers',fontsize=14)
for i, v in enumerate(bike_customers_by_region.values):
    bg.text(i, v + 1, str(v), ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()
```



In [23]:
```python
bike_customers = data_cleaned[data_cleaned['Purchased Bike']=='Yes']
bike_customers_by_gender = bike_customers['Gender'].value_counts()
print("Genderwise customers:")
print(bike_customers_by_gender)
```

```
Genderwise customers:
Gender
M    242
F    239
Name: count, dtype: int64
```

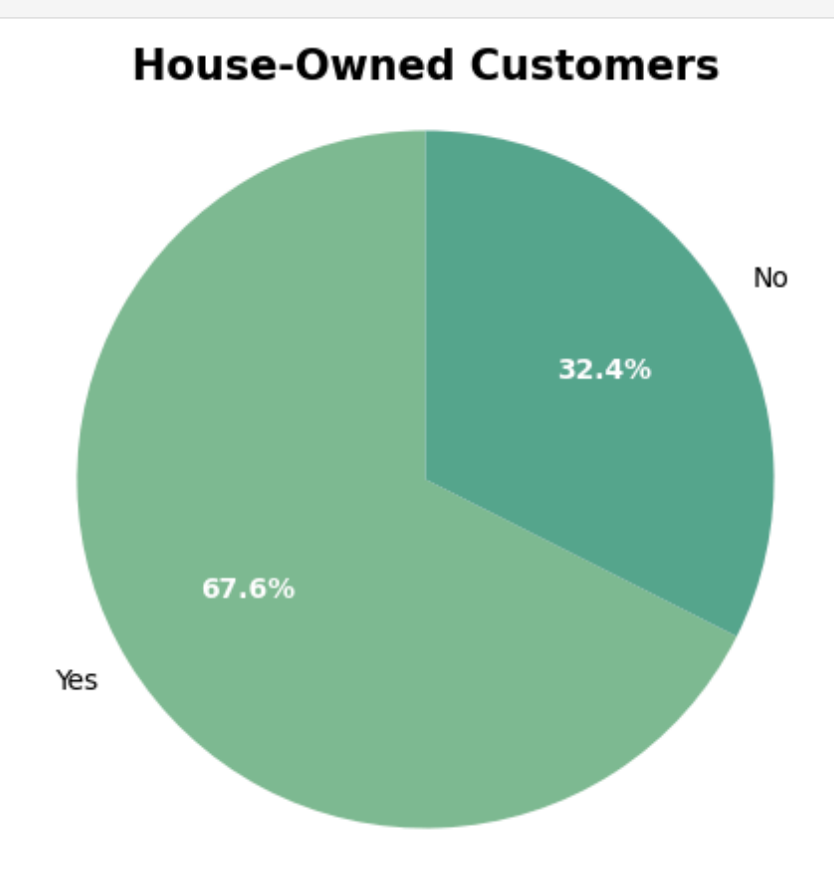
In [24]:
```python
plt.figure(figsize=(10,4))
bg = sns.barplot(x=bike_customers_by_gender.index,y=bike_customers_by_gender.values,color="red")
plt.title("Number of bike purchased customers by gender",fontsize=15,fontweight="bold")
plt.xlabel('Gender',fontsize=14)
plt.ylabel('Number of customers',fontsize=14)
for i, v in enumerate(bike_customers_by_gender.values):
    bg.text(i, v + 1, str(v), ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()
```



In [25]:
```python
bike_customers = data_cleaned[data_cleaned['Purchased Bike']=='Yes']
bike_customers_by_house = bike_customers['Home Owner'].value_counts()
print("House-Owned customers:")
print(bike_customers_by_house)
```

```
House-Owned customers:
Home Owner
Yes    325
No     156
Name: count, dtype: int64
```

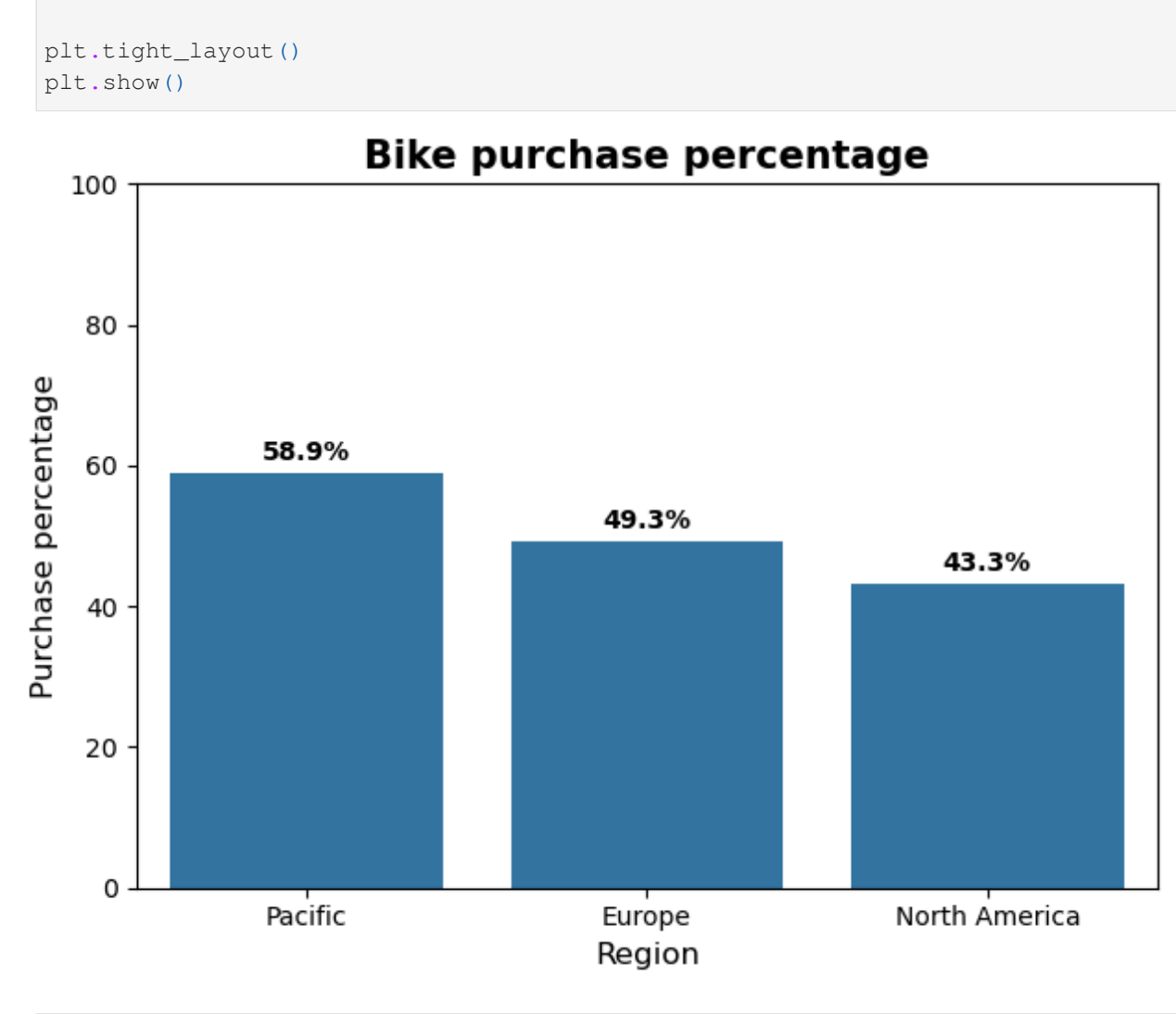
In [26]:
```python
colors = sns.color_palette('crest')[0:len(bike_customers_by_house)]
plt.figure(figsize=(10,5))
wedges,texts,autotexts = plt.pie(bike_customers_by_house.values,labels=bike_customers_by_house.index,colors=colors,autopct="%1.1f%%",startangle=90)
for autotext in autotexts:
    autotext.set_color('white')
    autotext.set_fontweight('bold')
plt.title("House-Owned Customers",fontsize=15,fontweight='bold')
plt.axis('equal')
plt.show()
```



In [31]:
```python
region_purchase_chat = pd.crosstab(index=data_cleaned['Region'],columns=data_cleaned['Purchased Bike'])
region_purchase_chat['Total_customers']= region_purchase_chat.sum(axis=1)
region_purchase_chat['Purchase_percentage']=(region_purchase_chat['Yes']/region_purchase_chat['Total_customers'])*100
region_purchase_summary = region_purchase_chat[['Yes','No','Total_customers','Purchase_percentage']].sort_values('Purchase_percentage',ascending=False)
print(region_purchase_summary)
```

```
Purchased Bike  Yes   No  Total_customers  Purchase_percentage
Region                                                        
Pacific         113   79              192            58.854167
Europe          148  152              300            49.333333
North America   220  288              508            43.307087
```

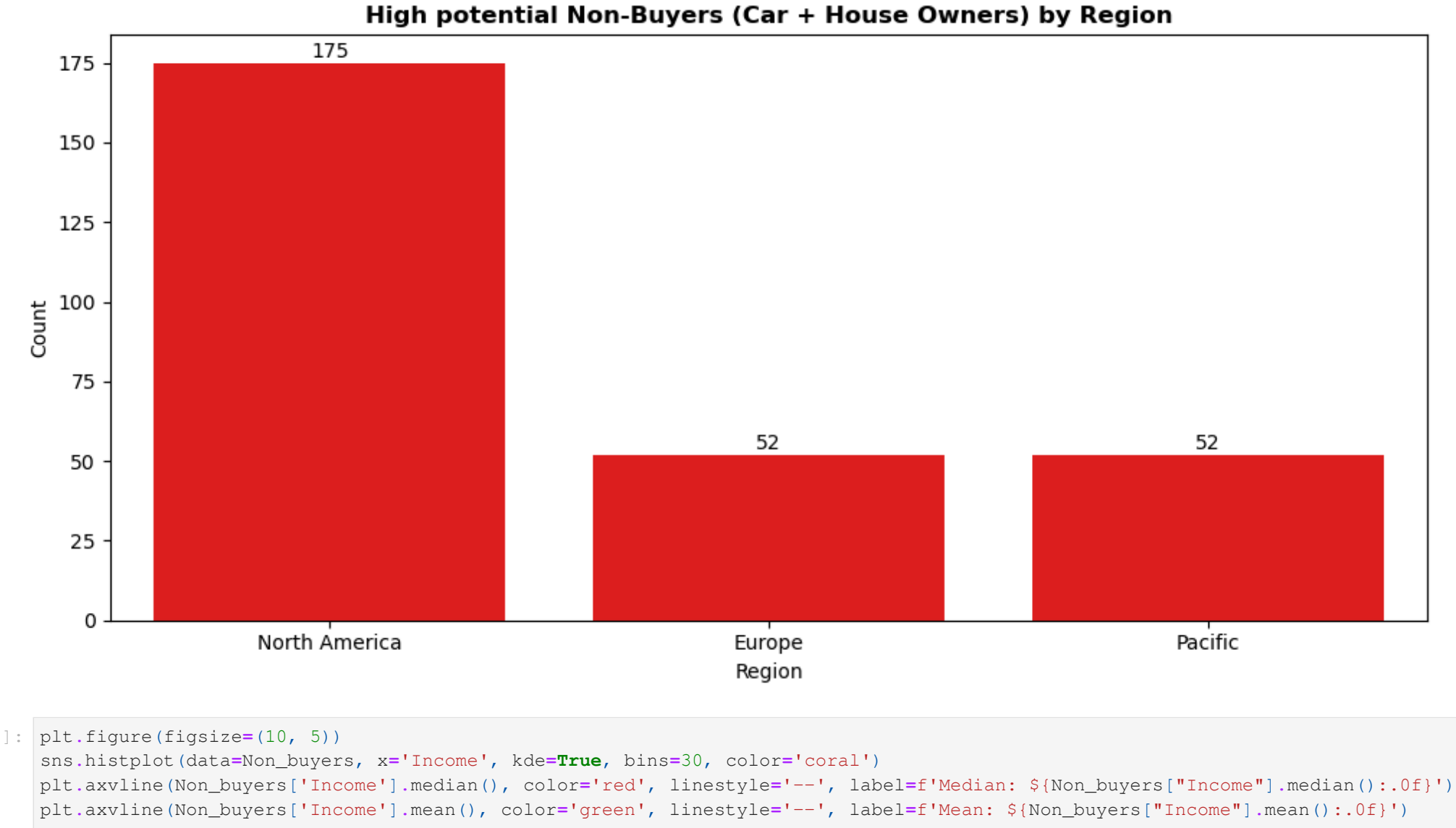
In [34]:
```python
bg = sns.barplot(x=region_purchase_summary.index,y=region_purchase_summary['Purchase_percentage'],order=region_purchase_summary.index)
plt.title("Bike purchase percentage",fontsize=15,fontweight='bold')
plt.xlabel("Region",fontsize=12)
plt.ylabel("Purchase percentage",fontsize=12)
plt.ylim(0,100)
for i, v in enumerate(region_purchase_summary['Purchase_percentage']):
    bg.text(i, v + 1, f'{v:.1f}%', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()
```



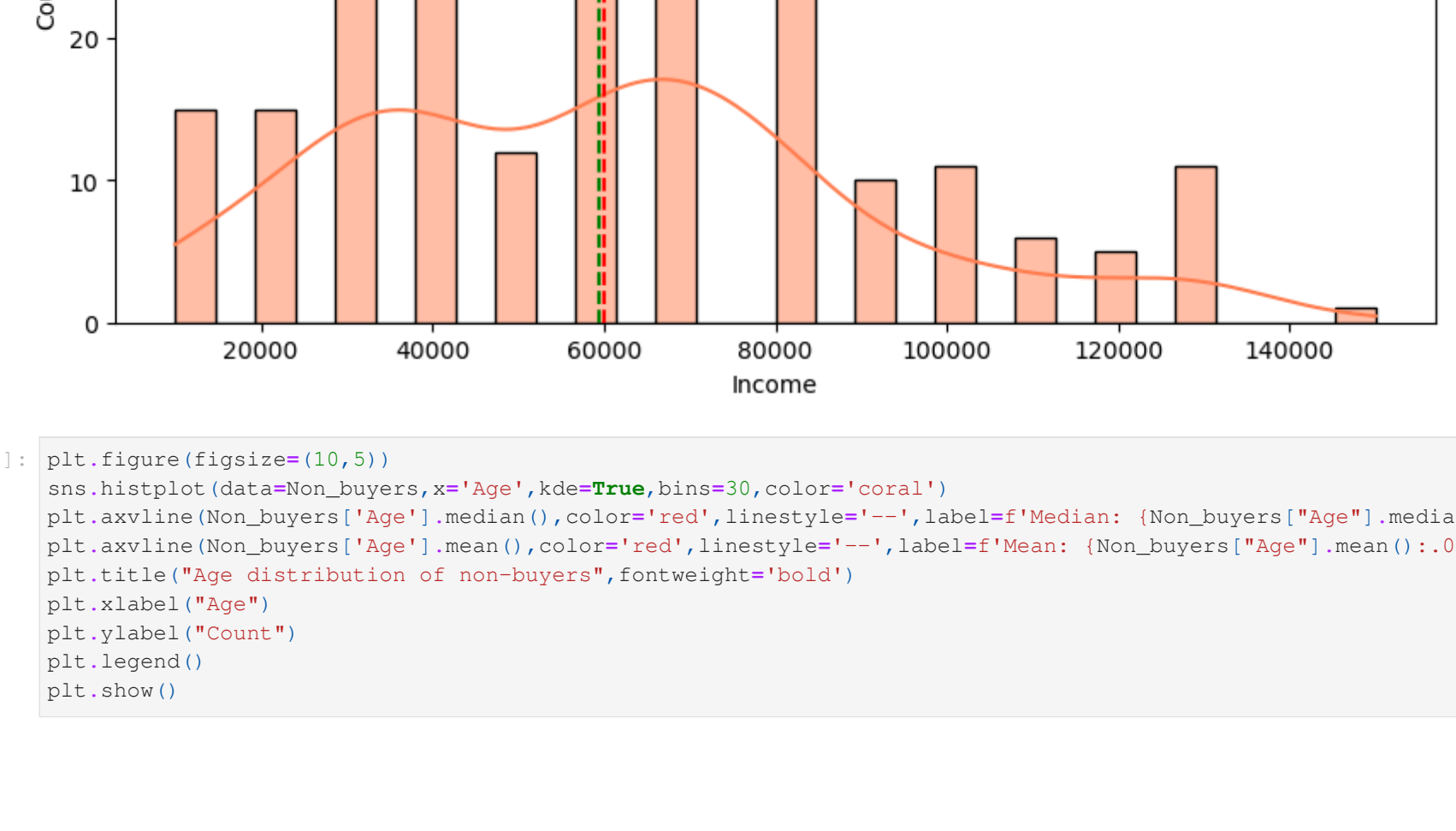
In [35]:
```python
Non_buyers = data_cleaned[(data_cleaned['Purchased Bike']=='No')&(data_cleaned['Cars']>=1)&(data_cleaned['Home Owner']=='Yes')]
print(f"Number of non-buyers with car and house:{len(Non_buyers)}")
```

```
Number of non-buyers with car and house:279
```

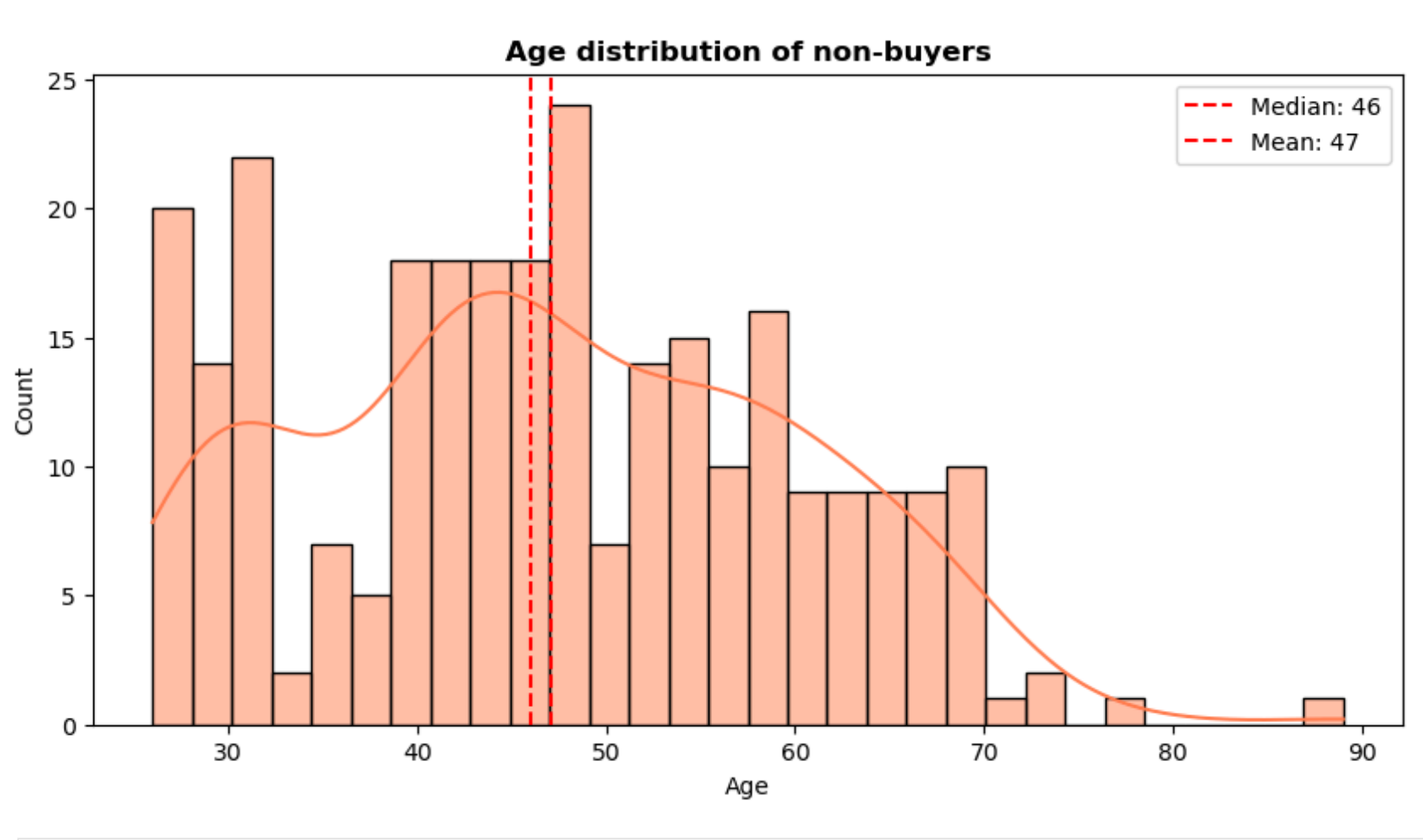
In [38]:
```python
plt.figure(figsize=(10, 5))
region_counts = Non_buyers['Region'].value_counts()
bg = sns.barplot(x=region_counts.index, y=region_counts.values,color='red')

plt.title('High potential Non-Buyers (Car + House Owners) by Region',fontweight='bold')
plt.xlabel('Region')
plt.ylabel('Count')

for i, v in enumerate(region_counts.values):
    bg.text(i, v + 0.2, str(v), ha='center', va='bottom')
plt.tight_layout()
plt.show()
```



In [41]:
```python
plt.figure(figsize=(10, 5))
sns.histplot(data=Non_buyers, x='Income', kde=True, bins=30, color='coral')
plt.axvline(Non_buyers['Income'].median(), color='red', linestyle='--', label=f'Median: ${Non_buyers["Income"].median():.0f}')
plt.axvline(Non_buyers['Income'].mean(), color='green', linestyle='--', label=f'Mean: ${Non_buyers["Income"].mean():.0f}')

plt.title('Income Distribution of High-Potential Non-Buyers', fontweight='bold')
plt.xlabel('Income')
plt.ylabel('Count')
plt.legend()
plt.show()
```



In [43]:
```python
plt.figure(figsize=(10,5))
sns.histplot(data=Non_buyers,x='Age',kde=True,bins=30,color='coral')
plt.axvline(Non_buyers['Age'].median(),color='red',linestyle='--',label=f'Median: {Non_buyers["Age"].median():.0f}')
plt.axvline(Non_buyers['Age'].mean(),color='red',linestyle='--',label=f'Mean: {Non_buyers["Age"].mean():.0f}')
plt.title("Age distribution of non-buyers",fontweight='bold')
plt.xlabel("Age")
plt.ylabel("Count")
plt.legend()
plt.show()
```


Age distribution of non-buyers
Median: 46
Mean: 47
Count
0
5
10
15
20
25
Age
30
40
50
60
70
80
90

```
In [ ]:
```